# Aide-mémoires

Toute la syntaxe et toutes les instructions de Lexis+, domaine par domaine, avec leurs enchaînements et les renvois entre domaines.




*Ne payez que ce dont vous avez réellement besoin !*

LexiWorks
Suite Collaborative

wdCommerce
Gestion Commerciale

LexiPilot
Décisionnel (B.I.)

CaSchd
Supervision Infra

# Toute la syntaxe en une page

Les blocs du langage, leurs mots d'ouverture, de suite et de fermeture, la priorité des opérateurs et les types.

100 mots-clés et opérateurs

## Lignes, valeurs, variables

```
&& commentaire   // commentaire   * ligne de commentaire
locale total = 0, nom = "Ada"      && locale
compteur += 1                       && globale implicite
x = a + ;                           && « ; » : suite à la ligne
    b
? "Total :", total   ?? "sans fin de ligne"
```

## Conditions

```
si cond
  …
sinonsi cond      (0..n)
sinon             (0..1)
finsi
faire cas / cas cond / autrement / fincas
```

## Boucles

```
pour i = 1 a 10 [pas 2]   …  suivant
pour chaque v dans liste   …  suivant
pour chaque k, v dans fiche …  suivant
tantque cond        …  fintantque
sortir   boucler     dans toute boucle
```

## Procédures

```
procedure NOM(a, b)     = fonction
  parametres x, y       (style dBase)
  locale r = a * b
retourner r               ou finprocedure
NOM(1, 2)   faire NOM avec 1, 2   @NOM
```

## Tables et objets

```
t = {1, 2, nom = "Ada"}   t[1]  t.nom
inserer v dans t [position p | clef "K"]
retirer de t position p dans x
nouvelle table t   supprimer table t
C = {}  C.__index = C
C:procedure P comme m   obj:m()
```

## Erreurs

```
essayer
  …
capturer e
  ? "erreur :", e
finessayer
erreur "message"   erreur {code = 404}
```

## SQL

```
sql "insert into t values (?, ?)" avec a, b
sql "select * from t where x > ?" avec 5 dans l
requete(sql, …)   valeursql(sql, …)
```

## Zones (dBase)

```
utiliser clients alias c ordre "nom" filtre "…"
parcourir pour ville = "Brest" … finparcourir
aller haut|bas|n  sauter n  chercher v
localiser pour cond   continuer
ajouter avec c par v  remplacer c par v  effacer
compter pour cond dans n  selectionner a  fermer tout
```

## Événements

```
ui = paquet("interface")
fen = ui.fenetre{titre = "…"}
b = fen:ajouter(ui.bouton("OK"))
b.surclic = @CLIC
ui.executer()
procedure CLIC(ev) … retourner
```

## Priorité des opérateurs

| De la plus forte… | Opérateurs |
|---|---|
| 1 | `^ **` (de droite à gauche) |
| 2 | `- non #` (unaires) |
| 3 | `* / %` |
| 4 | `+ - ..` |
| 5 | `= == <> != # < <= > >= $` |
| 6 | `et` |
| 7 …à la plus faible | `ou` |

## Types — type(v)

| | |
|---|---|
| `nul` | absence de valeur (faux dans un si) |
| `logique` | vrai, faux |
| `nombre` | 42, 3.14 — 0 est vrai ! |
| `chaine` | "…" — "" est vrai ! |
| `table` | liste et / ou fiche |
| `procedure` | @NOM |
| `objet · tache` | fournis par les paquets |



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



# Aide-mémoire · Démarrer

Écrire, exécuter, déboguer et lancer un programme.

Voir aussi : Les bases · Système · Procédures

21 instructions et fonctions

**ENCHAÎNEMENTS** `écrire principal.prg` → `Ctrl+R` → `console` · `F9 point d'arrêt` → `F5` → `F11 / F10` → `variables` · `wdlexisrt prog.prg a b` → `systeme.args` → `systeme.quitter(code)`

## Modèle type

```
&& principal.prg
? "Bonjour", saisir("Nom ? ")
pour i = 1 a 3
  ? i, CARRE(i)
suivant

fonction CARRE(n)
retourner n * n
```

## Afficher et saisir

**`? v1, v2, …`**
affiche, séparé par des espaces, puis passe à la ligne

**`?? v1, v2, …`**
affiche sans passer à la ligne

**`? (seul)`**
passe à la ligne

**`saisir([invite])`**
Lit une ligne saisie (console de l'atelier ou terminal)

**`paquet(nom)`**
Charge un paquet : interface, fichier, json, csv, http, reseau, tache

## Projet

**`principal.prg`**
fichier principal : son en-tête est le programme principal

**`autres .prg`**
procédures disponibles dans tout le projet (sous-dossiers compris)

**`procedure PRINCIPAL`**
exécutée si le fichier principal ne contient que des procédures

**`finprocedure / finfonction`**
fin explicite d'une procédure : la suite revient au programme principal

## Atelier wdlexis+

**`Ctrl+R`**
exécuter

**`F5`**
déboguer / continuer

**`F9`**
point d'arrêt

**`F11 · F10 · Maj+F11`**
pas à pas · pas principal · sortir

**`Maj+F5`**
arrêter

**`Ctrl+Espace`**
complétion

**`Ctrl+B`**
commenter

**`F12`**
aller à la définition

## Runtime wdlexisrt

**`wdlexisrt prog.prg [args]`**
exécute

**`--verifier prog.prg`**
vérifie la syntaxe

**`-e "code"`**
exécute une ligne

**`--paquets`**
liste les paquets

## Pièges à éviter

⚠ Le programme principal s'écrit en tête : tout ce qui suit une procédure lui appartient.

⚠ Un seul fichier peut contenir des instructions en tête : principal.prg.

⚠ Ctrl+R exécute le document tel qu'il est à l'écran, même non enregistré.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Les bases

Instructions, commentaires, valeurs, opérateurs et variables.

Voir aussi : Décider & répéter · Texte & calcul · Tables & objets

32 instructions et fonctions

**ENCHAÎNEMENTS** `"12,5"` → `nombre()` → `12.5` → `chaine(n, l, d)` → `" 12.50"` — `locale x` → `x = …` → `x += …` → `liberer x`

## Modèle type

```
locale prix = nombre(saisir("Prix ? "))
si prix = nul
  ? "Ce n'est pas un nombre"
sinon
  ? "TTC :", chaine(prix * 1.2, 1, 2)
finsi
? nom ou "inconnu"      && valeur par défaut
```

## Lignes et commentaires

**une instruction par ligne**
fin de ligne = fin d'instruction

**… ;**
continue sur la ligne suivante

**&& · // · ***
commentaires

**casse**
noms insensibles à la casse ; mots-clés en minuscules

**à = a**
accent facultatif

## Valeurs

**nul**
absence de valeur

**vrai faux .v. .f.**
logiques

**42 3.14 1e3**
nombres

**"…" '…' [[…]]**
chaînes (\n \t \" \\)

**{…}**
table

**@NOM**
référence de procédure

## Opérateurs

**+ - * / % ^ ****
calcul

**+ (avec texte) ..**
concaténation

**= ==**
égal

**<> != #**
différent

**< <= > >=**
comparaison

**$**
inclusion de texte

**et ou non**
logique (renvoient une des valeurs)

**#v**
longueur

## Variables

**locale x [= v], y**
variable locale

**globale x**
globale explicite

**x = v**
affectation (nom non déclaré = globale)

**x += v · x -= v**
ajouter, retirer

**stocker v dans a, b**
même valeur partout

**liberer x**
remet à nul

**SESSION**
table des variables globales

**m->x**
la variable x (et non un champ)

## Types et conversions

**type(v)**
"nul", "logique", "nombre", "chaine", "table", "procedure", "objet" ou "tache"

**chaine(v, [largeur], [decimales])**
Texte d'une valeur

**texte(v)**
Texte d'une valeur (utilise __tostring)

**valeur(s)**
Nombre d'un texte (virgule décimale admise), 0 si impossible (VAL)

**nombre(s)**
Nombre d'un texte, nul si impossible

## Pièges à éviter

- ⚠ 2 + "3" donne "23" : le + colle dès qu'il y a du texte.
- ⚠ "a" = "A" est faux : les textes respectent la casse.
- ⚠ 0 et "" sont VRAIS dans un si ; seuls nul et faux sont faux.
- ⚠ -2 ^ 2 vaut -4 ; 2 ^ 3 ^ 2 vaut 512.
- ⚠ Sans locale, une variable de procédure est globale.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



# Aide-mémoire · Décider & répéter

Choisir un chemin, répéter un traitement.

Voir aussi : Les bases · Tables & objets · Erreurs

16 instructions et fonctions

**ENCHAÎNEMENTS**

`si` → `sinonsi *` → `sinon ?` → `finsi`

`faire cas` → `cas *` → `autrement ?` → `fincas`

`pour / pour chaque` → `boucler · sortir` → `suivant`

`tantque` → `boucler · sortir` → `fintantque`

## Modèle type

```
pour chaque c dans clients
  si c.retard = 0
    boucler
  sinonsi c.retard > 30
    ? "À relancer :", c.nom
  finsi
suivant
tantque qte = nul ou qte <= 0
  qte = nombre(saisir("Qté ? "))
fintantque
```

## Conditions

**`si cond … finsi`**
exécute si la condition est vraie

**`sinonsi cond`**
autre condition (ou « sinon si »)

**`sinon`**
aucune condition vraie

**`faire cas … fincas`**
choix multiple

**`cas cond`**
une branche

**`autrement`**
tous les autres cas

**`sisinon(condition, si_vrai, si_faux)`**
Valeur selon la condition

**`vide(v)`**
Vrai pour nul, faux, 0, une chaîne blanche ou une table sans élément (EMPTY)

**`estnul(v)`**
Vrai si v vaut nul

## Boucles

**`pour i = a a b [pas p] … suivant`**
compteur (a / à / jusqua ; finpour)

**`pour chaque v dans liste … suivant`**
chaque valeur

**`pour chaque k, v dans fiche`**
chaque clef et valeur

**`pour chaque v dans @ITERATEUR`**
procédure appelée jusqu'à nul

**`tantque cond … fintantque`**
tant que (ou faire tantque)

**`sortir`**
quitte la boucle

**`boucler`**
passe au tour suivant

## Pièges à éviter

⚠ pour se ferme par suivant, tantque par fintantque, si par finsi.

⚠ Les bornes d'un pour sont calculées une seule fois.

⚠ Un tantque dont la condition reste vraie ne s'arrête jamais (Maj+F5).

⚠ Un itérateur renvoie nul pour arrêter la boucle.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



# Aide-mémoire · Procédures

Nommer un traitement pour le réutiliser partout.

Voir aussi : Tables & objets · Multitâche · Interface graphique

13 instructions et fonctions

**ENCHAÎNEMENTS** `procedure NOM(a)` → `parametres ?` → `retourner v` → `finprocedure ?` — `@NOM` → `appeler(@NOM, …) · ref(…)` → `trier · surclic · tache.lancer`

## Modèle type

```
? TTC(100), TTC(100, 5.5)

fonction TTC(ht, taux)
  taux = taux ou 20
retourner arrondi(ht * (1 + taux /
100), 2)

trier(liste, @PLUS_GRAND)
fonction PLUS_GRAND(a, b)
retourner a > b
```

## Définir

**`procedure NOM(a, b)`**
début d'une procédure

**`fonction NOM(a, b)`**
synonyme

**`parametres a, b`**
lire les arguments (style dBase)

**`retourner [valeur]`**
fin et résultat (aussi « retour »)

**`finprocedure · finfonction`**
fin explicite

**`argument manquant`**
vaut nul : x = x ou défaut

## Appeler

**`NOM(a, b)`**
appel

**`x = NOM(a, b)`**
appel avec résultat

**`faire NOM avec a, b`**
appel style dBase

**`@NOM`**
référence (sans appeler)

**`ref(a, b)`**
appel d'une référence rangée dans une variable

**`appeler(@proc, ...)`**
Appelle une procédure référencée

**`récursivité`**
une procédure s'appelle elle-même (200 niveaux au plus)

## Pièges à éviter

⚠ NOM appelle, @NOM désigne sans appeler.

⚠ Les tables passées en argument sont partagées (copier() pour une copie).

⚠ Les gestionnaires d'événements ne voient que les globales.

⚠ Toujours un cas d'arrêt dans une fonction récursive (200 niveaux).



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



# Aide-mémoire · Tables & objets

Listes, fiches, objets et métatables.

Voir aussi : Procédures · Fichiers & formats · Texte & calcul

33
instructions et fonctions

**ENCHAÎNEMENTS** `{ }` → `inserer · retirer` → `trier` → `joindre · json` — `Classe = {}` → `__index` → `:procedure … comme` → `fixermetatable` → `obj:methode()`

## Modèle type

```
locale l = {"b", "a"}
inserer "c" dans l
trier(l)
? joindre(l, ", "), #l
locale f = {nom = "Ada", age = 36}
f.ville = "Brest"
pour chaque k, v dans f
  ? k, v
suivant
```

## Créer et accéder

**`{a, b, c}`**
liste (indices à partir de 1)

**`{cle = v}`**
fiche

**`t[i] · t.cle · t["CLE"]`**
accès

**`nouvelle table t`**
t = {}

**`supprimer table t`**
t = nul

**`#t`**
longueur de la liste

## Modifier

**`inserer v dans t [position p]`**
ajoute

**`inserer v dans t clef "K"`**
ajoute sous une clef

**`retirer de t position p dans x`**
enlève

**`inserer(t, v, [position])`**
Ajoute v (à la fin ou à la position)

**`retirer(t, [position])`**
Enlève et renvoie un élément (le dernier par défaut)

**`trier(t, [@comparer])`**
Trie la liste sur place

**`copier(t)`**
Copie de premier niveau

## Interroger

**`taille(t)`**
Nombre d'éléments de la partie liste (#t)

**`nbelements(t)`**
Nombre total d'éléments

**`clefs(t)`**
Liste des clefs

**`valeurs(t)`**
Liste des valeurs

**`aclef(t, k)`**
Vrai si la clef existe

**`contient(s, sous)`**
Vrai si s contient sous

**`joindre(liste, [separateur])`**
Texte des éléments d'une liste

**`decouper(s, [separateur])`**
Liste des morceaux (séparateur « , » par défaut)

**`json(v)`**
Texte JSON d'une valeur

**`dejson(s)`**
Valeur d'un texte JSON (clefs d'objets en majuscules)

## Objets et métatables

**`Classe.__index = Classe`**
méthodes partagées

**`Classe:procedure P comme m`**
range une méthode

**`obj:m(…)`**
appel de méthode (obj en premier argument)

**`fixermetatable t, mt`**
forme instruction

**`fixermetatable(t, mt)`**
Donne une métatable

**`obtenirmetatable(t)`**
La métatable ou nul

**`lirebrut(t, k)`**
Lecture sans métaméthode

**`ecrirebrut(t, k, v)`**
Écriture sans métaméthode

**`__index __newindex __call __tostring __len`**
accès, appel, texte, longueur

**`__eq __lt __le __add __sub __mul __div __mod __pow __unm __concat`**
opérateurs

## Pièges à éviter

- ⚠ Les indices commencent à 1.
- ⚠ b = a ne copie pas : b et a sont la même table.
- ⚠ f.nom = f.NOM = f["NOM"] mais f["nom"] est une autre clef.
- ⚠ Une table vide s'encode {} en JSON.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Texte & calcul

La boîte à outils toujours disponible : texte, nombres, dates, sécurité.

Voir aussi : Les bases · Système · Tables & objets

55 instructions et fonctions

**ENCHAÎNEMENTS** `texte` → `decouper()` → `liste` → `joindre()` → `texte` | `hachermdp(mdp)` → `empreinte en base` → `verifiermdp(saisie, empreinte)`

## Modèle type

```
locale l = "2026-10-16;DUR-0042;1250,80"
locale c = decouper(l, ";")
? gauche(c[1], 4), droite(c[2], 4)
? valeur(c[3]) * 1.2
? chaine(3.14159, 8, 2), arrondi(2.345, 2)
? hachermdp("secret") <> nul
```

## Texte

**majuscule(s)**
Texte en majuscules (accents compris)

**minuscule(s)**
Texte en minuscules

**rogner(s)**
Retire les espaces aux deux bouts (ALLTRIM)

**rognerg(s)**
Retire les espaces à gauche (LTRIM)

**rognerd(s)**
Retire les espaces à droite (RTRIM)

**longueur(s)**
Nombre de caractères (ou d'éléments d'une liste)

**souschaine(s, debut, [n])**
Extrait n caractères à partir de debut (SUBSTR)

**gauche(s, n)**
Les n premiers caractères

**droite(s, n)**
Les n derniers caractères

**position(sous, s)**
Position de sous dans s, 0 si absente (AT)

**espace(n)**
n espaces (SPACE)

**repliquer(s, n)**
s répété n fois (REPLICATE)

**remplacertexte(s, a, b)**
Remplace toutes les occurrences de a par b (STRTRAN)

**contient(s, sous)**
Vrai si s contient sous

**commencepar(s, debut)**
Vrai si s commence par debut

**finitpar(s, fin)**
Vrai si s finit par fin

**decouper(s, [separateur])**
Liste des morceaux (séparateur « , » par défaut)

**joindre(liste, [separateur])**
Texte des éléments d'une liste

**chaine(v, [largeur], [decimales])**
Texte d'une valeur

**html(s)**
Échappe < > & " ' pour le HTML

**url(s)**
Encode un texte pour une URL

**car(code, ...)**
Caractères à partir de leurs codes Unicode

**code(s, [i])**
Code Unicode du i-ème caractère

## Nombres

**abs(x)**
Valeur absolue

**entier(x)**
Partie entière, vers zéro (INT)

**arrondi(x, [decimales])**
Arrondi (0 décimale par défaut)

**plancher(x)**
Entier inférieur

**plafond(x)**
Entier supérieur

**racine(x)**
Racine carrée

**mod(a, b)**
Reste de la division, du signe du diviseur

**max(a, b, ...)**
Plus grande valeur (nombres ou chaînes)

**min(a, b, ...)**
Plus petite valeur

**aleatoire([n])**
Réel entre 0 et 1, ou entier entre 1 et n

**sin(x)**
Sinus (radians)

**cos(x)**
Cosinus

**tan(x)**
Tangente

**asin(x)**
Arc sinus

**acos(x)**
Arc cosinus

**atan(y, [x])**
Arc tangente (atan2 avec deux arguments)

**exp(x)**
Exponentielle

**log(x, [base])**
Logarithme (népérien par défaut)

**pi()**
π

## Dates

**date()**
Date du jour, AAAA-MM-JJ

**heure()**
Heure, HH:MM:SS

**maintenant()**
Date et heure

**horodatage()**
Secondes depuis le 1er janvier 1970 (avec décimales)

## Tests

**type(v)**
"nul", "logique", "nombre", "chaine", "table", "procedure", "objet" ou "tache"

**estnul(v)**
Vrai si v vaut nul

**vide(v)**
Vrai pour nul, faux, 0, une chaîne blanche ou une table sans élément (EMPTY)

**sisinon(condition, si_vrai, si_faux)**
Valeur selon la condition

## Sécurité

**hachage(s)**
Empreinte SHA-256 en hexadécimal (pas pour les mots de passe)

**hachermdp(mdp)**
Empreinte PBKDF2-SHA256 salée d'un mot de passe

**verifiermdp(mdp, empreinte)**
Vérifie un mot de passe (temps constant)

**mdpaleatoire([n])**
Mot de passe aléatoire (16 caractères par défaut)

**uuid()**
Identifiant unique aléatoire

## Pièges à éviter

⚠ 0.1 + 0.2 ne vaut pas exactement 0.3 : arrondissez les montants.

⚠ longueur() compte des caractères, pas des octets.

⚠ hachage() n'est pas fait pour les mots de passe : hachermdp().

⚠ html() et url() pour tout texte venu de l'utilisateur.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Erreurs

Signaler, intercepter et traiter les erreurs.

Voir aussi : Bases de données · Procédures · Décider & répéter

12 instructions et fonctions

**ENCHAÎNEMENTS**

`essayer` → `instructions` → `capturer e` → `finessayer`

`erreur …` → `remonte la pile` → `premier capturer`

`begin transaction` → `essayer` → `commit / rollback`

## Modèle type

```
essayer
  DEBITER(article, qte)
capturer e
  ? "Refus :", e
finessayer

procedure DEBITER(a, q)
  si q <= 0
    erreur "quantité invalide"
  finsi
retourner
```

## Intercepter

**`essayer`**
début du bloc surveillé

**`capturer [e]`**
e reçoit l'erreur (texte ou table)

**`finessayer`**
fin du bloc

## Signaler

**`erreur "message"`**
lève une erreur

**`erreur {code = …, message = …}`**
erreur structurée

**`relancer`**
erreur ... dans capturer

**`systeme.quitter(code)`**
arrêt volontaire

## Messages fréquents

**`division par zéro`**
diviseur nul

**`indexation d'une valeur nulle`**
t vaut nul

**`procédure ou fonction inconnue`**
faute de frappe, fichier absent

**`nombre attendu, chaine reçu`**
calcul sur un texte

**`profondeur d'appel maximale`**
récursivité sans fin

## Pièges à éviter

⚠ Un capturer vide masque les problèmes : affichez ou journalisez.

⚠ L'erreur remonte jusqu'au premier essayer, même loin dans les appels.

⚠ Une erreur non interceptée arrête le programme (code 1).



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Bases de données

DuckDB intégré (SQL et zones dBase), SQLite, Firebird.

Voir aussi : Erreurs · Fichiers & formats · Tables & objets

60 instructions et fonctions

**ENCHAÎNEMENTS**

`ouvrirbase()` → `sql "…" avec … dans l` → `pour chaque r dans l`

`utiliser` → `parcourir [pour]` → `finparcourir` → `fermer`

`chercher · localiser` → `trouve()` → `continuer`

`sqlite.ouvrir · firebird.connecter` → `db:executer · db:requete` → `db:transaction(@P)` → `db:fermer()`

## Modèle type

```
ouvrirbase(":memoire:")
sql "create table c(code varchar,
nom varchar)"
sql "insert into c values (?, ?)"
avec "C1", "Ada"
pour chaque r dans requete("select
* from c")
  ? r.code, r.nom
suivant
utiliser c ordre "nom"
parcourir
  ? numenr(), nom
finparcourir
```

## Base DuckDB

**`ouvrirbase(chemin)`**
Ouvre une base DuckDB (":memoire:" pour une base en mémoire)

**`fermerbase()`**
Ferme la base et les zones

**`base()`**
Chemin de la base ouverte ("" si aucune)

**`requete(sql, ...)`**
Liste des lignes d'une requête DuckDB (paramètres « ? »)

**`valeursql(sql, ...)`**
Première valeur de la première ligne, ou nul

**`sql "…" [avec v, …] [dans liste]`**
exécute du SQL (paramètres ?)

**`begin transaction · commit · rollback`**
transactions (via sql)

## Zones (dBase)

**`utiliser t [alias a] [ordre "…"] [filtre "…"]`**
ouvre une zone (USE)

**`selectionner a`**
zone courante (SELECT)

**`fermer [a | tout]`**
ferme (CLOSE)

**`aller haut | bas | n`**
se positionner (GO)

**`sauter [n]`**
avancer (SKIP)

**`chercher v`**
recherche sur l'ordre (SEEK)

**`localiser pour cond`**
LOCATE

**`continuer`**
suivant (CONTINUE)

**`ajouter [vide | avec c par v, …]`**
APPEND

**`remplacer c par v, …`**
REPLACE

**`effacer`**
DELETE

**`parcourir [pour cond] … finparcourir`**
SCAN

**`compter [pour cond] dans x`**
COUNT

**`rafraichir`**
relit la zone

**`alias->champ · m->var`**
champ d'une zone · variable

**`fdf([alias])`**
Fin de fichier de la zone (EOF)

**`ddf([alias])`**
Début de fichier (BOF)

**`numenr([alias])`**
Numéro de l'enregistrement courant (RECNO)

**`nbenr([alias])`**
Nombre d'enregistrements (RECCOUNT)

**`trouve([alias])`**
Résultat du dernier chercher / localiser (FOUND)

**`alias()`**
Alias de la zone courante

**`champ(nom, [alias])`**
Valeur d'un champ par son nom (FIELD)

**`enregistrement([alias])`**
L'enregistrement courant sous forme de table

## SQLite

**`sqlite.ouvrir(chemin)`**
Ouvre (ou crée) une base

**`sqlite.version()`**
Version de la bibliothèque SQLite

**`db:executer(sql, ...)`**
Exécute une instruction avec paramètres « ? »

**`db:requete(sql, ...)`**
Liste des lignes

**`db:premier(sql, ...)`**
Première ligne ou nul

**`db:lignes(sql, ...)`**
Itérateur sur les lignes : pour chaque r dans db:lignes(sql)

**`db:dernierid()`**
Identifiant de la dernière ligne insérée

**`db:modifications()`**
Lignes modifiées par la dernière instruction

**`db:transaction(@proc)`**
Appelle proc(db) dans une transaction (annulée en cas d'erreur)

**`db:fermer()`**
Ferme la base

## Firebird

**`firebird.connecter(base, utilisateur, motdepasse[, options])`**
Connexion : "serveur:/chemin/base.fdb", "serveur/3050:alias" ou fichier local

**`firebird.creer(base, utilisateur, motdepasse[, {taillepage=8192, jeu="UTF8"}])`**
Crée une base puis s'y connecte

**`firebird.version()`**
Version de la bibliothèque cliente

**`firebird.disponible()`**
Vrai si la bibliothèque cliente est trouvée

**`db:executer(sql, ...)`**
Exécute une instruction avec paramètres « ? »

**`db:requete(sql, ...)`**
Liste des lignes

**`db:premier(sql, ...)`**
Première ligne ou nul

**`db:valeur(sql, ...)`**
Première colonne de la première ligne ou nul

**`db:lignes(sql, ...)`**
Itérateur sur les lignes : pour chaque r dans db:lignes(sql)

**`db:modifications()`**
Lignes modifiées par le dernier executer()

**`db:sequence(nom[, pas=1])`**
Valeur suivante d'une séquence (générateur)

**`db:transaction(@proc[, options])`**
Appelle proc(db) dans une transaction (validée, ou annulée en cas d'erreur)

**`db:debut([{isolation="lecturevalidee lecture=faux, attente=vrai|faux|secondes}])`**
Commence une transaction explicite

**`db:valider()`**
Valide la transaction explicite

**`db:annuler()`**
Annule la transaction explicite

**`db:entransaction()`**
Vrai si une transaction explicite est en cours

**`db:tables()`**
Tables de l'utilisateur

**`db:colonnes(table)`**
Colonnes d'une table : {nom=, type=, longueur=, echelle=, obligatoire=}

**`db:serveur()`**
Version du serveur

**`db:fermer()`**
Ferme la connexion (une transaction explicite non validée est annulée)

## Pièges à éviter

⚠ Jamais de valeurs collées dans le SQL : paramètres ? et avec.

⚠ update / delete sans where modifient TOUTES les lignes.

⚠ Après parcourir, la zone est en fin de fichier : aller haut.

⚠ Les colonnes se lisent comme des noms : r.nom = r.NOM.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Fichiers & formats

Lire et écrire fichiers, dossiers, CSV, JSON et binaire.

Voir aussi : Bases de données · Tables & objets · Système

54 instructions et fonctions

**ENCHAÎNEMENTS**

`es.ouvrir(c, mode)` → `f:lire · f:ecrire` → `f:fermer()`

`csv.lire` → `liste de fiches` → `csv.ecrire`

`json.lire` → `table` → `json.ecrire`

`binaire.coder(fmt, …)` → `octets` → `binaire.decoder(fmt, d)`

## Modèle type

```
pour chaque l dans
es.lignes("journal.log")
  ? l
suivant
locale t =
csv.lire("ventes.csv", {sep =
";", entete = vrai, convertir =
vrai})
json.ecrire("ventes.json", t)
fichier.ajouter("journal.log",
date() + car(10))
```

## fichier

**fichier.lire(chemin)**
Contenu complet d'un fichier (texte ou binaire)

**fichier.lireoctets(chemin)**
Contenu binaire (les chaînes sont des suites d'octets)

**fichier.ecrire(chemin, donnees)**
Remplace le contenu d'un fichier

**fichier.ecrireoctets(chemin, donnees)**
Écrit des octets

**fichier.ajouter(chemin, donnees)**
Ajoute à la fin d'un fichier

**fichier.lignes(chemin)**
Liste des lignes d'un fichier

**fichier.ecrirelignes(chemin, lignes)**
Écrit une liste de lignes

**fichier.existe(chemin)**
Vrai si le fichier ou le dossier existe

**fichier.estfichier(chemin)**
Vrai pour un fichier

**fichier.estdossier(chemin)**
Vrai pour un dossier

**fichier.taille(chemin)**
Taille en octets

**fichier.liste([dossier], [recursif])**
Noms contenus dans un dossier (les dossiers finissent par /)

**fichier.creerdossier(chemin)**
Crée un dossier (et ses parents)

**fichier.supprimer(chemin)**
Supprime un fichier ou un dossier (récursif)

**fichier.copier(source, destination)**
Copie (dossiers récursivement)

**fichier.deplacer(source, destination)**
Déplace ou renomme

**fichier.infos(chemin)**
Table {taille, genre ("fichier" ou "dossier"), modifie, cree} ou nul

**fichier.courant()**
Dossier courant

**fichier.changer(dossier)**
Change de dossier courant

**fichier.joindre(a, b, ...)**
Assemble des chemins

**fichier.nom(chemin)**
Nom du fichier

**fichier.dossier(chemin)**
Dossier parent

**fichier.extension(chemin)**
Extension

**fichier.absolu(chemin)**
Chemin absolu

**fichier.dossiertemp()**
Dossier temporaire

**fichier.dossierperso()**
Dossier personnel

## es

**es.ecrire(...)**
Écrit les valeurs (sans séparateur ni fin de ligne) sur la sortie : console...

**es.lire([format])**
Lit l'entrée : "l" une ligne (défaut), "n" un nombre, "L" une ligne avec sa...

**es.lignes([chemin])**
Itérateur sur les lignes d'un fichier (ou de l'entrée) : pour chaque l dans...

**es.ouvrir(chemin, [mode])**
Ouvre un fichier : "l" lecture (défaut), "e" écriture, "a" ajout, "l+", "e+"

**es.type(v)**
"fichier", "fichier fermé" ou nul

**es.sortie**
Sortie standard

**es.erreur**
Sortie d'erreur

**es.entree**
Entrée standard

**f:lire([format])**
Lit : "l", "L", "n", "t" (tout) ou un nombre d'octets

**f:ecrire(...)**
Écrit des valeurs

**f:lignes()**
Itérateur sur les lignes restantes

**f:position([origine], [decalage])**
Déplace la position ("debut", "courant", "fin") et la renvoie

**f:vider()**
Vide les tampons

**f:fermer()**
Ferme le fichier

## csv

**csv.analyser(texte, [options])**
Liste des lignes d'un texte CSV

**csv.lire(chemin, [options])**
Lit un fichier CSV (mêmes options)

**csv.encoder(lignes, [options])**
Texte CSV

**csv.ecrire(chemin, lignes, [options])**
Écrit un fichier CSV

## json

**json.encoder(valeur, [joli])**
Texte JSON

**json.decoder(texte, [exact])**
Valeur Lexis+

**json.lire(chemin, [exact])**
Lit et décode un fichier JSON

**json.ecrire(chemin, valeur, [joli])**
Encode et écrit un fichier JSON (indenté par défaut)

**json.nul**
Valeur représentant null dans les données à encoder

## binaire

**binaire.coder(format, ...)**
Code des valeurs en binaire

**binaire.decoder(format, donnees, [position])**
Décode

**binaire.taille(format)**
Taille d'un format fixe

**binaire.hex(donnees)**
Représentation hexadécimale

**binaire.dehex(hex)**
Octets à partir de l'hexadécimal

## Pièges à éviter

⚠ fichier.supprimer() est définitif et récursif.

⚠ fichier.joindre() pour construire les chemins.

⚠ json.decoder met les clefs en majuscules, sauf exact = vrai.

⚠ es.lignes() pour les gros fichiers (pas de chargement complet).



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Système

Dates, environnement, commandes, arguments.

Voir aussi : Texte & calcul · Fichiers & formats · Démarrer

22 instructions et fonctions

**ENCHAÎNEMENTS**

`systeme.temps({…})` → `systeme.ecart()` → `/ 86400 = jours`

`systeme.capturer(cmd)` → `{sortie, code}`

`systeme.args` → `traitement` → `systeme.quitter(code)`

## Modèle type

```
? systeme.date("%A %d %B %Y")
locale n = systeme.temps({annee = 2026, mois
= 12, jour = 25})
? arrondi(systeme.ecart(n, systeme.temps())
/ 86400), "jours"
locale r = systeme.capturer("ls")
si #systeme.args < 1
  systeme.quitter(2)
finsi
```

## systeme

**`systeme.temps([date])`**
Heure actuelle (secondes Unix), ou date d'une table {annee, mois, jour,...

**`systeme.horloge()`**
Temps écoulé depuis le lancement, en secondes (horloge monotone)

**`systeme.date([format], [t])`**
Date formatée : %Y %m %d %H %M %S %A (jour) %B (mois) %F %T %x

**`systeme.ecart(t2, t1)`**
Différence en secondes

**`systeme.env(nom)`**
Variable d'environnement (nul si absente)

**`systeme.fixerenv(nom, valeur)`**
Définit une variable d'environnement

**`systeme.executer(commande)`**
Exécute une commande système

**`systeme.capturer(commande)`**
Exécute une commande

**`systeme.supprimer(chemin)`**
Supprime un fichier ou un dossier

**`systeme.renommer(ancien, nouveau)`**
Renomme un fichier

**`systeme.nomtemporaire()`**
Nom de fichier temporaire

**`systeme.quitter([code])`**
Termine le programme

**`systeme.attendre(secondes)`**
Pause (les fenêtres restent réactives)

**`systeme.plateforme()`**
"linux", "macosx", "windows"

**`systeme.dossierprogramme()`**
Dossier du programme principal

**`systeme.args`**
Arguments du programme

## Formats de systeme.date

**`%Y %m %d`**
année, mois, jour

**`%H %M %S`**
heure, minute, seconde

**`%A %a %B %b`**
jour, mois (en lettres)

**`%F %T %x %X %c`**
formats composés

**`%j %w %y %Z %%`**
jour de l'année, de la semaine...

**`"*t" "!…"`**
fiche de date ; heure UTC

## Pièges à éviter

⚠ Une journée ne fait pas toujours 86 400 s (changement d'heure) : arrondissez.

⚠ Les commandes du système diffèrent entre Linux, macOS et Windows.

⚠ Les secrets se lisent dans l'environnement : systeme.env().



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Réseau

HTTP, TCP, UDP, FTP, SSH et SFTP.

Voir aussi : Multitâche · Fichiers & formats · Système

41 instructions et fonctions

**ENCHAÎNEMENTS**

`http.requete{…}` → `{statut, corps, entetes}` → `dejson(corps)`

`reseau.ecouter(port)` → `s:accepter()` → `c:recevoir · c:envoyer` → `c:fermer()`

`reseau.connecter(h, p)` → `s:envoyer · s:recevoir` → `s:fermer()`

`ssh.connecter{…}` → `s:executer(cmd)` → `{sortie, code}`

## Modèle type

```
locale r = http.requete{url =
"https://api.exemple.fr/x", delai = 10}
si r.statut = 200
  locale d = dejson(r.corps)
finsi
locale s = reseau.connecter("srv", 7000)
s:envoyer("PING\n")
? s:recevoir()
s:fermer()
```

## http

**`http.lire(url, [entetes])`**
Requête GET

**`http.envoyer(url, corps, [entetes])`**
Requête POST

**`http.requete{methode=, url=, entetes=, corps=, delai=}`**
Requête quelconque

**`http.telecharger(url, chemin)`**
Télécharge dans un fichier

**`http.encoder(texte | table)`**
Encodage d'URL (une table devient a=1&b=2)

**`http.decoder(texte)`**
Décodage d'URL

## reseau

**`reseau.connecter(hote, port, [options])`**
Connexion TCP

**`reseau.ecouter(port, [hote])`**
Socket serveur TCP

**`reseau.udp([port], [hote])`**
Socket UDP, éventuellement lié à un port

**`reseau.resoudre(hote)`**
Adresse IP d'un nom

**`reseau.nomhote()`**
Nom de la machine

## Socket (s:)

**`s:envoyer(donnees)`**
Envoie des octets

**`s:recevoir([mode])`**
Reçoit : "l" une ligne (défaut), "t" tout jusqu'à la fermeture, ou un nombre…

**`s:delai(secondes)`**
Délai maximal des réceptions (nul = aucun)

**`s:accepter()`**
Attend un client (socket serveur)

**`s:envoyerversers(donnees, port, hote)`**
Envoi UDP

**`s:recevoirde([taille])`**
Réception UDP : {donnees, adresse, port}

**`s:distant()`**
{adresse, port} de l'autre extrémité

**`s:fermer()`**
Ferme le socket

## ftp

**`ftp.connecter{hote=, port=21, utilisateur="anonymous", motdepasse=""}`**
Ouvre une session FTP (mode passif)

**`c:liste([chemin], [detail])`**
Noms des fichiers (detail = vrai : format détaillé)

**`c:lire(distant, [local])`**
Télécharge : renvoie les données, ou les écrit dans le fichier local

**`c:envoyer(local_ou_donnees, [distant], [sontdesdonnees])`**
Envoie un fichier local (ou des données)

**`c:dossier(chemin)`**
Change de dossier distant

**`c:courant()`**
Dossier distant courant

**`c:creerdossier(chemin)`**
Crée un dossier

**`c:supprimerdossier(chemin)`**
Supprime un dossier

**`c:supprimer(chemin)`**
Supprime un fichier

**`c:renommer(ancien, nouveau)`**
Renomme

**`c:fermer()`**
Ferme la session

## ssh

**`ssh.connecter{hote=, utilisateur=, port=22, clef=, motdepasse=, options={}}`**
Ouvre une session (vérifie la connexion)

**`ssh.executer{hote=, utilisateur=, commande=, ...}`**
Exécute une commande sans garder de session : {sortie, code}

**`session:executer(commande)`**
Exécute une commande distante : {sortie, code}

**`session:lire(distant, [local])`**
Télécharge un fichier ou dossier (scp)

**`session:envoyer(local, [distant])`**
Envoie un fichier ou dossier (scp)

**`session:liste([chemin])`**
Noms d'un dossier distant (sftp)

**`session:creerdossier(chemin)`**
Crée un dossier distant (sftp)

**`session:supprimerdossier(chemin)`**
Supprime un dossier distant (sftp)

**`session:supprimer(chemin)`**
Supprime un fichier distant (sftp)

**`session:renommer(ancien, nouveau)`**
Renomme (sftp)

## sftp

**`sftp.connecter{hote=, utilisateur=, port=22, clef=, motdepasse=}`**
Ouvre une session de transfert (identique à ssh.connecter)

## Pièges à éviter

⚠ Les en-têtes de réponse sont en minuscules : r.entetes["content-type"].

⚠ Fixez un délai (delai = …) pour ne pas attendre indéfiniment.

⚠ Jamais de mot de passe dans le code : systeme.env() ou clef SSH.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



# Aide-mémoire · Multitâche

Tâches coopératives et fils d'exécution.

Voir aussi : Réseau · Procédures · Intelligence artificielle

22 instructions et fonctions

**ENCHAÎNEMENTS**

`tache.lancer(@P)` → `tache.executer()` → `t:attendre()`

`tache.canal()` → `c:envoyer(v)` → `c:recevoir()`

`fil.demarrer(code)` → `f:envoyer ⇄ fil.recevoir` → `fil.envoyer ⇄ f:recevoir` → `f:joindre()`

## Modèle type

```
tache.lancer(@MINUTEUR, "pâtes", 0.3)
tache.lancer(@MINUTEUR, "œufs", 0.2)
tache.executer()
locale f = fil.demarrer([[
fil.envoyer(fil.recevoir() * 2) ]])
f:envoyer(21)
? f:recevoir()
f:joindre()
```

## tache

**`tache.lancer(@proc, ...)`**
Crée une tâche qui exécutera proc(...)

**`tache.executer()`**
Fait tourner les tâches jusqu'à ce qu'elles soient toutes terminées

**`tache.attendre(secondes)`**
Dans une tâche : la suspend

**`tache.ceder()`**
Rend la main aux autres tâches

**`tache.canal([capacite])`**
Canal de communication (capacité 0 = illimitée)

**`tache.nombre()`**
Nombre de tâches en cours

**`tache.maintenant()`**
Heure en secondes (avec décimales)

**`t:attendre()`**
Attend la fin d'une tâche et renvoie son résultat

**`t:etat()`**
"suspendue", "en cours", "terminee" ou "erreur"

**`t:annuler()`**
Arrête une tâche suspendue

**`c:envoyer(valeur)`**
Envoie une valeur (attend si le canal est plein)

**`c:recevoir([delai])`**
Reçoit une valeur (attend si vide)

**`c:nombre()`**
Nombre de valeurs en attente

## fil

**`fil.demarrer(code_ou_fichier, ...)`**
Lance un fil qui exécute du code Lexis+ (chaîne) ou un fichier .prg, avec...

**`fil.envoyer(valeur)`**
Dans le fil : envoie une valeur au programme qui l'a lancé

**`fil.recevoir([delai])`**
Dans le fil : reçoit une valeur (nul après le délai)

**`fil.estenfant()`**
Vrai dans un fil lancé par fil.demarrer

**`f:envoyer(valeur)`**
Envoie une valeur au fil (tables, chaînes, nombres, logiques)

**`f:recevoir([delai])`**
Reçoit une valeur du fil

**`f:actif()`**
Vrai tant que le fil s'exécute

**`f:joindre()`**
Attend la fin du fil

**`f:arreter()`**
Demande l'arrêt du fil

## Pièges à éviter

⚠ Une attente bloquante (réseau, systeme.attendre) bloque toutes les tâches : utilisez un fil.

⚠ Un fil ne partage aucune variable : uniquement des messages.

⚠ Fichier de fil : sous-dossier, uniquement des procédures dont PRINCIPAL.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Interface graphique

Fenêtres wdgui : contrôles, événements, dialogues.

Voir aussi : Pages web · Procédures · Bases de données

154 instructions et fonctions

**ENCHAÎNEMENTS**

`ui = paquet("interface")` → `ui.fenetre{…}` → `fen:ajouter(ui.bouton(…))` → `btn.surclic = @P` → `ui.executer()`

`PROC(ev)` → `ev.genre · ev.x · ev.texte · ev.chemin` → `ev:arreter()`

## Modèle type

```
ui = paquet("interface")
fen = ui.fenetre{titre = "Démo",
largeur = 360, hauteur = 160}
lbl = fen:ajouter(ui.etiquette("",
"centre"))
btn = fen:ajouter(ui.bouton("OK",
{defaut = vrai}))
btn.surclic = @CLIC
ui.executer()
procedure CLIC(ev)
  lbl.libelle = "Cliqué à " +
heure()
retourner
```

## Fenêtre et boucle

**`interface.fenetre{titre=, largeur=, hauteur=, disposition=, theme=, sombre=}`**
Crée une fenêtre

**`interface.fenetres()`**
Nombre de fenêtres ouvertes

**`interface.executer()`**
Attend la fermeture des fenêtres (atelier) ou lance la boucle de wdgui…

**`interface.quitter()`**
Ferme toutes les fenêtres du programme

**`interface.fenetres()`**
Nombre de fenêtres ouvertes

**`interface.tracer([actif])`**
Affiche dans la console chaque événement reçu (diagnostic)

**`interface.separateur{vertical=, position=}`**
Séparateur (deux enfants)

**`interface.sep`**
Séparateur des chemins d'arbre et de menu

**`fen:fermer()`**
Ferme la fenêtre

**`fen:fixertheme(nom, [sombre])`**
Change le thème

**`c:ajouter(enfant, ...)`**
Ajoute des contrôles à un conteneur ou une fenêtre

**`c:focus()`**
Donne le focus

**`c:menucontextuel(element, ...)`**
Menu contextuel

**`c:redessiner()`**
Redessine le contrôle

## Contrôles

**`interface.etiquette(texte, [alignement])`**
Libellé (alignement : gauche, centre, droite)

**`interface.bouton(libelle, [{defaut=, annuler=}])`**
Bouton

**`interface.saisie(texte | {texte=, indication=, multiligne=})`**
Champ de saisie

**`interface.cases(option, ...)`**
Cases à cocher

**`interface.radio(option, ...)`**
Boutons radio

**`interface.liste([multi])`**
Liste

**`interface.combo(element, ...)`**
Liste déroulante

**`interface.tableau()`**
Tableau simple

**`interface.arbre()`**
Arbre

**`interface.grille{multi=, remplissage="programme|manuel|automa`**
Grille de saisie

**`interface.jauge()`**
Jauge

**`interface.curseur()`**
Potentiomètre

**`interface.plage()`**
Potentiomètre de plage

**`interface.ascenseur()`**
Ascenseur

**`interface.compteur()`**
Compteur à flèches

**`interface.notation()`**
Notation

**`interface.image(chemin)`**
Image

**`interface.codebarre(texte)`**
Code-barres

**`interface.graphe([genre], [titre])`**
Graphe : colonnes, lignes, aires, secteurs

**`interface.calendrier(["AAAAMMJJ"])`**
Calendrier

**`interface.kanban()`**
Kanban

**`interface.mosaique()`**
Mosaïque proportionnelle (treemap)

**`interface.gantt()`**
Gantt

**`interface.barremenu()`**
Barre de menus

**`interface.onglets()`**
Onglets

**`interface.separateur{vertical=, position=}`**
Séparateur (deux enfants)

**`interface.conteneur([disposition], [colonnes])`**
Conteneur

**`interface.colonne()`**
Conteneur vertical

**`interface.rangee()`**
Conteneur horizontal

**`interface.superconteneur([dispositic`**
Conteneur sans marge

**`interface.cadre(titre, [disposition])`**
Cadre de groupe

**`interface.panneau{disposition=, defilement=, colonnes=}`**
Conteneur défilant (defilement : toujours, automatique, masque)

**`interface.pdf()`**
Visionneuse PDF (PDFium)

## Dialogues

**`interface.alerte(message, [icone], [titre])`**
Message (icone : information, question, attention, stop, aucune)

**`interface.confirmer(message, [icone])`**
OK / Annuler

**`interface.demander(message, [defaut])`**
Saisie d'un texte

**`interface.ouvrirfichier{titre=, filtre=, dossier=}`**
Choix d'un fichier à ouvrir

**`interface.enregistrerfichier{nom=, filtre=}`**
Choix d'un fichier à enregistrer

**`interface.couleur(["#RRGGBB"])`**
Sélecteur de couleur

**`interface.police([famille], [taille])`**
Sélecteur de police : {famille, taille, gras, italique}

## Listes, tableaux, arbres

**`liste:ajouter(element, ...)`**
Ajoute (listes et combos)

**`liste:inserer(i, element)`**
Insère

**`liste:supprimer(i)`**
Supprime

**`liste:vider()`**
Vide

**`liste:selectionner(i)`**
Sélectionne

**`liste:selection()`**
Index sélectionné

**`liste:nombre()`**
Nombre d'éléments

**`liste:element(i)`**
Texte d'un élément

**`liste:modifier(i, texte)`**
Modifie

**`liste:chercher(texte)`**
Recherche

**`liste:deselectionner(i)`**
Désélectionne

**`tableau:ajoutercolonne(titre, [largeur], [alignement])`**
Ajoute une colonne

**`tableau:ajouterligne(cellule, ...)`**
Ajoute une ligne

**`tableau:cellule(ligne, colonne)`**
Texte d'une cellule



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

**tableau:fixercellule(ligne, colonne, texte)**
Modifie une cellule

**tableau:selection()**
Ligne sélectionnée

**tableau:trier(colonne, [croissant])**
Trie

**tableau:insererligne(i, cellule, ...)**
Insère une ligne

**tableau:ligne(i)**
Cellules d'une ligne

**tableau:nombre()**
Nombre de lignes

**tableau:selectionner(i)**
Sélectionne une ligne

**tableau:supprimer(i)**
Supprime une ligne

**tableau:vider()**
Vide le tableau

**arbre:ajouter(chemin...)**
Ajoute un nœud ("A", "B" ou "A/B")

**arbre:deplier(chemin...)**
Déplie

**arbre:replier(chemin...)**
Replie

**arbre:supprimer(chemin...)**
Supprime

**arbre:vider()**
Vide

**arbre:estdeplie(chemin...)**
Vrai si le nœud est déplié

**arbre:selectionner(chemin...)**
Sélectionne un nœud

**arbre:nombre()**
Nombre de nœuds

**onglets:ajouterpage(titre, [disposition])**
Ajoute une page

**barremenu:ajouter(chemin...)**
Ajoute un élément de menu ("Fichier", "Ouvrir")

**cases:coche([i])**
État d'une case

**cases:cocher(i, [b])**
Coche une case

**cases:ajouteroption(texte)**
Ajoute une case

**radio:ajouteroption(texte)**
Ajoute un bouton radio

## Grille

**grille:ajoutercolonne{nom=, titre=, genre=, largeur=, choix=, decimales=}**
Ajoute une colonne (genre : texte, nombre, date, image, bouton, combo, case)

**grille:ajouterligne(cellule, ...)**
Ajoute une ligne

**grille:cellule(ligne, colonne)**
Valeur d'une cellule (colonne = nom ou index)

**grille:fixercellule(ligne, colonne, valeur)**
Modifie une cellule

**grille:nombre()**
Nombre de lignes

**grille:trier(colonne, [croissant])**
Trie

**grille:chercher(colonne, valeur, [exact], [debut])**
Cherche une ligne

**grille:fixerpied(colonne, genre)**
Pied de colonne : somme, moyenne, min, max, nombre

**grille:ajouterformat(colonne, operateur, valeur, {fond=, texte=, ligneentiere=})**
Mise en forme conditionnelle

**grille:fixermodifiable([b])**
Mode saisie

**grille:verscsv([sep])**
Export CSV

**grille:chargercsv(chemin, [sep])**
Import CSV

**grille:versjson()**
Export JSON

**grille:chargerjson(chemin)**
Import JSON

**grille:insererligne(i, ...)**
Insère une ligne

**grille:modifierligne(i, ...)**
Remplace une ligne

**grille:supprimerligne(i)**
Supprime une ligne

**grille:vider()**
Vide la grille

**grille:ligne(i)**
Cellules d'une ligne

**grille:nombrecolonnes()**
Nombre de colonnes

**grille:colonne(col)**
Objet colonne

**grille:selection([i])**
Ligne sélectionnée (i-ème)

**grille:nombreselection()**
Nombre de lignes sélectionnées

**grille:selectionner(l)**
Sélectionne une ligne

**grille:deselectionner(l)**
Désélectionne

**grille:toutselectionner()**
Sélectionne tout

**grille:trouver(texte)**
Cherche partout : {ligne, colonne}

**grille:valeurpied(col, [genre])**
Valeur d'un pied de colonne

**grille:effacerformats()**
Retire les mises en forme

**grille:fixerrupture([col])**
Regroupe sur une colonne

**grille:fixermodecolonne(col, mode)**
Saisie d'une colonne (lecture, modifiable)

**grille:fixermodeligne(l, mode)**
Saisie d'une ligne

**grille:fixerremplissage(mode)**
programme, manuel, automatique

**grille:fixerlibellebouton(col, texte)**
Libellé des boutons d'une colonne

**grille:modifiercellule(l, col)**
Place la saisie dans une cellule

**grille:decsv(texte, [ajout])**
Remplit depuis un texte CSV

**grille:enregistrercsv(chemin, [sep])**
Écrit un CSV

**grille:enregistrerjson(chemin)**
Écrit un JSON

## Graphes, planning, PDF

**graphe:ajouterdonnee(serie, categorie, valeur)**
Ajoute une donnée

**graphe:libellecategorie(i, texte)**
Libellé de catégorie

**graphe:libelleserie(i, texte)**
Libellé de série

**graphe:fixergenre(genre)**
Genre de graphe

**calendrier:afficher(annee, mois)**
Affiche un mois

**kanban:ajouterliste(titre)**
Ajoute une liste ; renvoie son numéro

**kanban:ajouter(liste, texte)**
Ajoute une carte

**mosaique:ajouter(libelle, valeur)**
Ajoute une tuile proportionnelle

**gantt:ajoutertache(nom, debut, duree, [avancement])**
Ajoute une tâche ; renvoie son numéro

**gantt:fixeravancement(i, valeur)**
Avancement d'une tâche (%)

**panneau:defilera(x, y)**
Fait défiler

**panneau:defilerde(dx, dy)**
Fait défiler d'un décalage

**panneau:rendrevisible(controle)**
Amène un contrôle à l'écran

**pdf:ouvrir(chemin)**
Ouvre un PDF

**pdf:enregistrer([chemin])**
Enregistre

**pdf:nouveau([largeur], [hauteur])**
Nouveau document vide

**pdf:fermer()**
Ferme le document

**pdf:nombrepages()**
Nombre de pages

**pdf:page()**
Page courante

**pdf:allera(n)**
Va à la page n

**pdf:zoom(z)**
Zoom

**pdf:ajoutertexte(page, x, y, texte, [taille])**
Écrit sur une page

**pdf:importerpages(chemin, [plage], [position])**
Insère des pages d'un autre PDF

## Propriétés

**valeur texte num libelle titre visible etat actif infobulle nom x y largeur hauteur couleur fond bordure accent rayon coche min max ancrage poids largeurfixe hauteurfixe etirer marge espacement colonnes famille genre**
ctl.propriete en lecture et en écriture

## Événements

**surclic surdoubleclic surclicdroit surclicmilieu surdoubleclicdroit surboutonbas surboutonhaut surentree sursortie surdeplacement surmolette surtouchebas surtouchehaut sursaisie surfocus surfinfocus surchangement surselection surdeploiement surrepli survalidation surouverture surfermeture surredimension suractivation surdesactivation surevenement**
ctl.surXXX = @PROC ; PROC(ev) ; ev:arreter()

## Pièges à éviter

⚠ Listes, combos, radios, onglets : valeur = numéro choisi (à partir de 1).

⚠ Les gestionnaires ne voient que les globales.

⚠ ui.tracer(vrai) affiche chaque événement reçu.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Pages web

Pages HTML / CSS / JavaScript reliées à Lexis+.

Voir aussi : Interface graphique · Fichiers & formats · Tables & objets

31 instructions et fonctions

**ENCHAÎNEMENTS**

`webview.fenetre{fichier}` → `f:lier · f:autoriser` → `f.surpret = @PRET` → `f:executer()`

`f:appeler("fn", …, @R)` → `fn() en JS` → `R(valeur, erreur)` — `await nomjs(a)` → `PROC(a)` → `retourner v`

## Modèle type

```
f = webview.fenetre{titre = "Démo", fichier = "page.htm"}
f:lier("total", @TOTAL)          && JS : await total(2, 3)
f.surpret = @PRET
f:executer()
procedure PRET(ev)
  f:appeler("afficher", "Bonjour", @VU)
retourner
```

## webview

**`webview.fenetre{titre=, largeur=, hauteur=, taille=, fichier=, url=, html=, debogage=, console=}`**
Crée une fenêtre web

**`webview.disponible()`**
Vrai si la bibliothèque webview est chargée (sinon voir webview.erreur())

**`webview.erreur()`**
Raison de l'échec du chargement de la bibliothèque (nul si aucune)

**`webview.version()`**
Version de la bibliothèque webview

**`webview.bibliotheque()`**
Chemin de la bibliothèque chargée

**`webview.url(chemin)`**
URL file:// d'un fichier local (relatif au dossier du programme)

**`webview.js(valeur)`**
Littéral JavaScript (JSON) d'une valeur Lexis+

**`fen:html(code)`**
Affiche du HTML

**`fen:naviguer(url)`**
Charge une URL (https://, file://, data:...)

**`fen:ouvrir(chemin)`**
Charge un fichier HTML local (relatif au dossier du programme)

**`fen:initialiser(js)`**
Ajoute un script exécuté avant chaque page

**`fen:evaluer(code, [@rappel])`**
Lexis+ → JavaScript : évalue du code

**`fen:appeler(fonction, arg..., [@rappel])`**
Lexis+ → JavaScript : appelle window.fonction(arg...) (chemin "app.maj"...

**`fen:emettre(nom, [donnees])`**
Lexis+ → JavaScript : déclenche les écouteurs lexis.sur(nom, fn)

**`fen:lier(nomjs, [@procedure])`**
JavaScript → Lexis+ : crée window.nomjs(...) (Promise) qui appelle la...

**`fen:delier(nomjs)`**
Retire une liaison

**`fen:autoriser(nom, ...)`**
JavaScript → Lexis+ : procédures appelables par lexis.appeler(nom, ...)

**`fen:taille(largeur, hauteur, [genre])`**
Taille de la fenêtre (genre : normale, min, max, fixe)

**`fen:executer()`**
Affiche la fenêtre et traite ses événements jusqu'à sa fermeture

**`fen:fermer()`**
Ferme la fenêtre (termine fen:executer())

## Côté JavaScript

**`nomjs(…)`**
fonction liée par f:lier : Promise

**`lexis.appeler(nom, …)`**
procédure autorisée : Promise

**`lexis.envoyer(d)`**
→ f.surmessage

**`lexis.journal(…)`**
→ console Lexis+

**`lexis.sur(nom, fn) · lexis.retirer`**
écouteurs de f:emettre

**`lexis.pret`**
page chargée

## Propriétés

**`f.titre`**
titre (lecture / écriture)

**`f.surpret`**
page chargée : P(ev), ev.url

**`f.surmessage`**
lexis.envoyer(d) : P(d)

**`f.surfermeture`**
après la fermeture

**`f.prete · f.ouverte`**
états

## Pièges à éviter

- ⚠ Appelez la page dans surpret (les appels antérieurs attendent).
- ⚠ La réponse JS arrive dans un rappel (valeur, erreur).
- ⚠ macOS : lancez avec wdlexisrt (fil principal).
- ⚠ Évitez autoriser("*") si la page affiche du contenu externe.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

# Aide-mémoire · Intelligence artificielle

Modèles de langage, GGUF, dessin, audio, tenseurs, entraînement.

Voir aussi : Multitâche · Fichiers & formats · Pages web

174 instructions et fonctions

**ENCHAÎNEMENTS**

`llm.charger(chemin)` → `m:conversation{…}` → `c:demander(q)` → `réponse`

`entrainement.tokeniseur` → `entrainement.nouveau` → `m:entrainer` → `m:enregistrer` → `llm.charger`

`tenseur.parametre` → `perte:retropropager()` → `opt:etape()`

## Modèle type

```
m = llm.charger("modele.gguf")
c = m:conversation({systeme = "Sois
concis.", maxtokens = 200})
? c:demander("Résume ce texte.",
{pieces = {llm.joindre("doc.txt")}})
f = c:demanderjson("Trois villes ?",
[[{"villes": [str]}]])
? joindre(f.villes, ", ")
```

## llm

**`llm.charger(chemin, {fils=0, verbeux=faux})`**
Charge un modèle GGUF (fichier « mappé » : quasi instantané)

**`llm.version()`**
Version de la bibliothèque nimllm

**`llm.fils([n])`**
Nombre de fils de calcul (0 = tous les cœurs)

**`llm.echantillonnage([table])`**
Réglages d'échantillonnage par défaut (complétés par la table) :...

**`llm.glouton([table])`**
Réglages du décodage glouton (déterministe)

**`llm.echantillonneur([reglages])`**
Échantillonneur manuel : s:tirer(logits), s:accepter(id)

**`llm.joindre(chemin)`**
Pièce jointe lue sur le disque (texte, CSV, JSON, PDF, PNG/BMP, WAV...)

**`llm.joindretexte(nom, contenu)`**
Pièce jointe construite en mémoire

**`llm.joindredonnees(nom, octets)`**
Pièce jointe à partir d'octets (le type est déduit du nom et du contenu)

**`llm.extrairejson(texte)`**
Premier objet JSON trouvé dans un texte (table) ou nul

**`llm.sansbalises(texte)`**
Retire les balises ``` d'un bloc de code

**`llm.similarite(v1, v2)`**
Similarité cosinus de deux vecteurs (m:vecteur)

**`llm.prompt(gabarit, messages[, suite=vrai])`**
Texte d'un dialogue {{role=, contenu=}...} mis en forme (llama3, chatml,...

**`llm.gabarits()`**
Liste des gabarits de conversation

**`llm.tokeniseur(chemin)`**
Tokeniseur seul lu dans un fichier GGUF

**`llm.predecouper(texte[, mode])`**
Pré-découpage avant BPE (llama3, qwen2, gpt2)

**`m:conversation({systeme=, contexte=4096, maxtokens=512, echantillonnage=, gabarit="auto", langue="fr", arrets=})`**
Nouvelle conversation (le texte seul est accepté comme prompt système)

**`m:contexte([taille])`**
Contexte brut (cache KV) : evaluer, completer, generer

**`m:decrire()`**
Résumé du modèle

**`m:vecteur(texte)`**
Vecteur (embedding) d'un texte

**`m:perplexite(texte)`**
Perplexité d'un texte (plus elle est basse, plus le texte est attendu)

**`m:appliquerlora(chemin)`**
Applique un adaptateur LoRA (.lora.gguf) à la volée

**`m:retirerlora()`**
Retire l'adaptateur LoRA

**`m:fermer()`**
Libère le modèle

**`m.tokeniseur`**
Tokeniseur du modèle (propriétés : nom, architecture, couches, dimension,...

**`c:demander(question[, options | @flux])`**
Réponse (fiche texte, brut, fichiers, json, tokensprompt, tokensgeneres,...

**`c:demanderjson(question[, schema])`**
Réponse JSON convertie en table (clefs en majuscules)

**`c:regenerer([@flux])`**
Nouveau tirage de la dernière réponse

**`c:prolonger([maxtokens, @flux])`**
Prolonge la dernière réponse

**`c:generer([options | @flux])`**
Réponse de l'assistant à l'historique actuel (après c:ajouter)

**`c:ajouter(role, texte)`**
Ajoute un message (systeme, utilisateur, assistant) : exemples « few-shot »,...

**`c:annuler()`**
Retire le dernier échange

**`c:reinitialiser()`**
Oublie l'historique (garde le prompt système)

**`c:enregistrer(chemin)`**
Enregistre la conversation (JSON)

**`c:charger(chemin)`**
Recharge une conversation

**`c:etat()`**
État du contexte (tokens, messages, vitesse)

**`c:prompt([question])`**
Texte exact envoyé au modèle

**`c:tokensprompt()`**
Tokens de l'historique mis en forme

**`c.systeme`**
Propriété : systeme, maxtokens, arrets, echantillonnage, tailleimage, voix,...

**`x:evaluer(ids)`**
Ajoute des tokens au contexte et renvoie les logits du dernier

**`x:completer(texte[, options | @flux])`**
Complétion brute (sans gabarit de conversation)

**`x:generer(ids[, options | @flux])`**
Génère une suite à partir de tokens

**`x:reinitialiser()`**
Vide le contexte

**`x:tronquer(n)`**
Ne garde que les n premiers tokens

**`l:meilleurs([n])`**
Les n tokens les plus probables : {{id=, proba=}...}

**`l:valeur(id)`**
Logit d'un token

**`l:argmax()`**
Token le plus probable

**`s:tirer(logits)`**
Tire un token selon les réglages

**`s:accepter(id)`**
Mémorise un token (pénalités de répétition)

**`t:encoder(texte[, bos=faux, speciaux=vrai])`**
Liste des identifiants de tokens

**`t:decoder(ids[, speciaux=faux])`**
Texte d'une liste de tokens

**`t:morceau(id[, special=faux])`**
Texte d'un token

**`t:id(texte)`**
Identifiant d'un token (nul s'il n'existe pas)

**`t:estspecial(id)`**
Vrai pour un token spécial

**`t:enregistrer(chemin)`**
Enregistre le tokeniseur dans un fichier GGUF

**`t.taille`**
Propriétés : taille, bos, eos, pad, inconnu, fins, fusions, type, gabarit,...

**`p:texte([max])`**
Texte inséré dans le message (propriétés : nom, type, mime, contenu, taille,...

## gguf

**`gguf.ouvrir(chemin)`**
Ouvre un fichier GGUF (lecture seule, mappé en mémoire)

**`gguf.quantifiermodele(entree, sortie[, type="q4_k", garder_sortie=vrai])`**
Re-quantifie un modèle (f32, f16, bf16, q8_0, q6_k, q5_k, q5_0, q5_1, q4_k,...

**`gguf.fusionnerlora(base, adaptateur, sortie[, type="q8_0"])`**
Fusionne un adaptateur LoRA dans un nouveau modèle autonome

**`gguf.types()`**
Types de poids utilisables pour la quantification

**`gguf.bitsparpoids(type)`**
Nombre moyen de bits par poids d'un type

**`gguf.quantifier(type, valeurs)`**
Quantifie puis relit une liste : {octets=, valeurs=, erreur=}

**`g:decrire([max_tenseurs])`**
Résumé du fichier

**`g:existe(cle)`**
Vrai si la métadonnée existe

**`g:valeur(cle)`**
Valeur d'une métadonnée (nombre, texte, logique ou liste)

**`g:texte(cle[, defaut])`**
Métadonnée texte

**`g:entier(cle[, defaut])`**
Métadonnée entière

**`g:reel(cle[, defaut])`**
Métadonnée réelle

**`g:cles()`**
Liste des clefs de métadonnées

**`g:tenseurs()`**
Liste des tenseurs : {nom=, type=, dims=, elements=, octets=}

**`g:tenseur(nom[, max])`**
Valeurs d'un tenseur (déquantifiées)

**`g:fermer()`**
Ferme le fichier (propriétés : version, chemin, alignement, architecture)

## dessin

**`dessin.svg(code[, largeur, hauteur, fond])`**
Rastérise un dessin SVG

**`dessin.image(largeur, hauteur[, fond])`**
Image unie



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

**dessin.toile(largeur, hauteur[, fond, lissage=3])**
Surface de dessin anticrénelée

**dessin.lire(chemin)**
Lit une image BMP, PPM ou PNG

**dessin.extrairesvg(texte)**
Code SVG contenu dans un texte (réponse d'un modèle) ou nul

**dessin.taillesvg(code)**
Dimensions déclarées d'un SVG : {largeur=, hauteur=}

**dessin.couleur(texte | r, g, b[, a])**
Couleur sous forme de table {r=, g=, b=, a=}

**t:rectangle(x, y, largeur, hauteur, couleur)**
Rectangle plein

**t:cercle(cx, cy, rayon, couleur)**
Disque

**t:ellipse(cx, cy, rx, ry, couleur)**
Ellipse pleine

**t:polygone(points, couleur)**
Polygone plein

**t:ligne(x0, y0, x1, y1, epaisseur, couleur)**
Segment

**t:polyligne(points, epaisseur, couleur[, fermee])**
Ligne brisée

**t:terminer()**
Termine le dessin et renvoie l'image

**img:enregistrer(chemin)**
Enregistre selon l'extension (.bmp ou .ppm)

**img:enregistrerbmp(chemin)**
Enregistre en BMP

**img:enregistrerppm(chemin)**
Enregistre en PPM

**img:pixel(x, y)**
Couleur d'un pixel : {r=, g=, b=}

**img:fixerpixel(x, y, couleur)**
Colore un pixel

**img:decrire([largeur])**
Description textuelle (couleurs, aperçu en caractères)

## audio

**audio.parler(texte[, {langue="fr", hauteur=115, vitesse=1, frequence=16000}])**
Synthèse vocale (fr ou en)

**audio.melodie(notation[, bpm=120])**
Mélodie : « C4 E4 G4:2 R » (note+octave[:temps], R = silence)

**audio.nouveau([frequence])**
Son vide

**audio.lire(chemin)**
Lit un fichier WAV PCM 16 bits

**audio.infos(chemin)**
En-tête d'un WAV : {frequence=, canaux=, bits=, duree=}

**audio.phonemes(texte[, langue])**
Transcription phonétique utilisée par la synthèse

**audio.enlettres(n[, langue])**
Nombre écrit en toutes lettres

**audio.frequencenote(note)**
Fréquence d'une note (« A4 » = 440)

**s:enregistrer(chemin)**
Écrit un fichier WAV (16 bits mono)

**s:silence(secondes)**
Ajoute un silence

**s:ajouter(autre)**
Ajoute un autre son à la suite (rééchantillonné si besoin)

**s:normaliser([crete=0.9])**
Ajuste le volume

**s:copier()**
Copie du son (propriétés : duree, frequence, echantillons)

## tenseur

**tenseur.depuis(valeurs[, forme, gradient=faux, nom])**
Tenseur à partir d'une liste

**tenseur.plein(forme[, valeur=0, gradient=faux, nom])**
Tenseur rempli d'une valeur

**tenseur.zeros(forme[, gradient=faux, nom])**
Tenseur nul

**tenseur.scalaire(v[, gradient=faux])**
Tenseur d'une seule valeur

**tenseur.aleatoire(forme[, ecart=1, gradient=vrai, nom])**
Valeurs gaussiennes N(0, écart²)

**tenseur.parametre(forme[, ecart=0.1, nom])**
Paramètre entraînable initialisé au hasard

**tenseur.graine(n)**
Fixe la graine du générateur aléatoire du paquet

**tenseur.hasard([max | "gauss"])**
Nombre aléatoire du générateur du paquet

**tenseur.lineaire(x, w[, b])**
x · wᵀ + b (x : {n, entrées}, w : {sorties, entrées})

**tenseur.produit(a, b)**
Produit matriciel {n, k} · {k, m}

**tenseur.echelle(t, s)**
Multiplie par un nombre (aussi : t * s)

**tenseur.somme(t)**
Somme des éléments

**tenseur.moyenne(t)**
Moyenne des éléments

**tenseur.relu(t)**
max(0, x)

**tenseur.silu(t)**
x · sigmoïde(x)

**tenseur.sigmoide(t)**
1 / (1 + e^-x)

**tenseur.tanh(t)**
Tangente hyperbolique

**tenseur.gelu(t)**
GELU

**tenseur.softmax(t)**
Softmax par ligne

**tenseur.mse(prediction, cible)**
Erreur quadratique moyenne

**tenseur.entropiecroisee(logits, classes)**
Entropie croisée (classes : liste d'entiers à partir de 0

**tenseur.concatener(a, b)**
Concaténation des colonnes

**tenseur.decrochage(t, p)**
Dropout (entraînement)

**tenseur.sansgradient(@proc, ...)**
Appelle proc sans construire de graphe (inférence)

**tenseur.verifiergradient(@proc, t[, eps, echantillons])**
Écart relatif entre gradient calculé et différence finie

**tenseur.operation(parents, valeurs, forme, @retro)**
Opération personnalisée

**tenseur.gradientrequis(t, ...)**
Vrai si l'un des tenseurs demande un gradient

**tenseur.adamw(parametres[, {lr=3e-4, beta1=0.9, beta2=0.95, decroissance=0.1}])**
Optimiseur AdamW

**tenseur.sgd(parametres[, {lr=0.01, moment=0.9, decroissance=0}])**
Descente de gradient avec moment

**tenseur.lrcosinus(etape, echauffement, total, lrmax[, lrmin])**
Taux d'apprentissage (échauffement + cosinus)

**tenseur.normegradient(parametres)**
Norme globale des gradients

**t:retropropager()**
Calcule les gradients de tout le graphe (t scalaire)

**t:valeur([i=1])**
Valeur d'un élément

**t:donnees()**
Liste des valeurs

**t:gradient()**
Liste du gradient (nul s'il n'a pas été calculé)

**t:forme()**
Forme (liste des dimensions)

**t:taille()**
Nombre d'éléments

**t:zerograd()**
Remet le gradient à zéro

**t:detacher()**
Copie sans gradient

**t:remodeler(forme)**
Même données, autre forme

**t:ajoutergradient(valeurs)**
Ajoute au gradient (opérations personnalisées)

**o:zerograd()**
Remet à zéro les gradients des paramètres

**o:etape()**
Met à jour les paramètres

**o:ecreter(norme_max)**
Limite la norme des gradients

**o:enregistrer(chemin)**
Enregistre l'état (moments) de l'optimiseur

**o:charger(chemin)**
Recharge l'état

## entrainement

**entrainement.tokeniseur(textes[, {taille=2048, speciaux=, frequencemin=2, mode="llama3"}])**
Apprend un tokeniseur BPE niveau octet (compatible llama.cpp)

**entrainement.tokeniseuroctets([speciaux])**
Tokeniseur minimal (256 octets + spéciaux)

**entrainement.nouveau(tok, {dimension=256, couches=4, tetes=4, tetes_kv=, cache=, contexte=256, nom=, graine=42})**
Nouveau modèle Transformer (architecture Llama)

**entrainement.charger(chemin[, {mode="lora", rang=8, alpha=16, cibles={"q","k","v","o"}}])**
Charge un GGUF à spécialiser : LoRA (poids gelés) ou complet

**entrainement.textes(tok, texte[, {graine=, bos=vrai}])**
Données de pré-entraînement (texte continu)

**entrainement.fichiers(tok, chemins)**
Données de pré-entraînement lues dans des fichiers

**entrainement.dialogues(tok, gabarit, dialogues[, {systeme=}])**
Données de dialogue (seules les réponses sont apprises)

**entrainement.jsonl(chemin, tok[, gabarit="auto", systeme])**
Dialogues lus dans un fichier JSONL

**entrainement.gabarit(tok)**
Gabarit de conversation adapté au tokeniseur

**entrainement.configuration()**
Réglages d'entraînement par défaut

**m:entrainer(donnees[, {etapes, lot, sequence, accumulation, lr, lrmin, echauffement, decroissance, ecretage, evaluation, lotsevaluation, journal, sauvegarde, chemin, echantillon, amorce, graine, validation, optimiseur, rappel}])**
Entraîne le modèle

**m:parametres()**
Liste des tenseurs entraînables

**m:nombreparametres([tous=faux])**
Nombre de paramètres

**m:optimiseur([{lr=, decroissance=}])**
AdamW sur les paramètres du modèle

**m:enregistrer(chemin[, type="f32"])**
Exporte un vrai fichier GGUF (llm.charger, llama.cpp, Ollama)

**m:enregistrerlora(chemin)**
Enregistre l'adaptateur LoRA

**m:sauvegarder(optimiseur, prefixe)**
Point de sauvegarde (modèle ou adaptateur + optimiseur)

**m:evaluer(donnees[, lots=4, lot=8, sequence=64])**
Perte moyenne (exp(perte) = perplexité)

**m:generer(amorce[, {maxtokens=50, temperature=0.8}])**
Génère un texte (surveillance de l'entraînement)

**d:separer([fraction=0.1])**
Sépare en {entrainement, validation}

**d:taille()**
Nombre de tokens (texte) ou de dialogues

**d:pluslong()**
Longueur du plus long dialogue (tokens)

## Pièges à éviter

⚠ Compilez avec -d:release : 10 à 30 fois plus rapide.

⚠ c:reinitialiser() pour repartir d'une conversation vide.

⚠ Le contexte (en tokens) limite la mémoire de la conversation.

⚠ Les blocs de quantification font 32 valeurs.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr